

P14

Toolformer: los modelos de lenguaje pueden enseñarse a sí mismos a usar herramientas

Toolformer: Language Models Can Teach Themselves to Use Tools

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, y otros · 2023 · arXiv:2302.04761 · NeurIPS 2023

Ficha pedagógica del Artificial Intelligence Evolution Program. No es el paper original: es una guía de lectura en español que enlaza a la fuente primaria. Consultado 2026-08-16.

P14 — Toolformer

El modelo decide por sí solo cuándo le conviene llamar a una herramienta, usando como criterio su propia pérdida. Nadie etiqueta nada.

Nivel: L3 · Motor: toolformer · Notebook: P14_toolformer.ipynb

1. Identificación

Campo	Valor
Título original	Toolformer: Language Models Can Teach Themselves to Use Tools
Autoría	Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu y otros
Año	2023
Venue	arXiv:2302.04761 · NeurIPS 2023
Fuente primaria	arXiv:2302.04761
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

ReAct demostró que un modelo puede usar herramientas dentro de un bucle. Pero **qué herramientas existen y cuándo conviene llamarlas** venía escrito a mano en el prompt, o requería datos anotados por humanos.

Eso tiene dos problemas: anotar es caro, y lo anotado limita el modelo a las herramientas y situaciones que alguien previó. Además, los modelos de lenguaje fallan de forma sistemática y predecible en cosas que un programa de tres líneas resuelve: aritmética exacta, fechas, datos actualizados, traducción de términos raros.

La pregunta: **¿puede el modelo descubrir solo dónde le conviene pedir ayuda?**

3. Propuesta

Un procedimiento autosupervisado en cuatro pasos:

1. **Muestrear** posiciones candidatas del texto donde podría insertarse una llamada, y generar llamadas plausibles usando unos pocos ejemplos en el prompt.
2. **Ejecutar** cada llamada y obtener su resultado real.
3. **Filtrar**: conservar la llamada solo si el resultado **reduce la pérdida** de predecir el texto que viene después, por encima de un umbral τ .
4. **Reentrenar** el modelo sobre el corpus enriquecido con las llamadas que sobrevivieron.

La idea central es el criterio del paso 3: **la utilidad de una herramienta se mide por cuánto ayuda a predecir el texto siguiente**. Es una señal automática, disponible en cualquier corpus, sin ningún anotador.

Herramientas del artículo: calculadora, sistema de preguntas y respuestas, buscador, traductor y calendario.

4. Intuición sin fórmulas

Un estudiante con una calculadora sobre la mesa. Prueba a usarla en muchos sitios del examen y se queda con la costumbre solo donde le ayudó de verdad. Donde no, deja de usarla. Nadie le dice cuándo: lo deduce del resultado.

Dónde deja de funcionar la analogía: el estudiante sabe si la calculadora dio un número correcto. El modelo solo sabe si el resultado le **ayudó a predecir el texto siguiente**. Son cosas distintas: una herramienta que devuelve siempre lo mismo puede resultar «útil» por razones espurias.

5. Matemática mínima

Para una posición i , con llamada c y respuesta r :

$$\begin{aligned} L^+ &= L(x_{\{i:\}} \mid \text{prefijo} + [c \rightarrow r]) && \text{con el resultado de la herramienta} \\ L^- &= \min(L(x_{\{i:\}} \mid \text{prefijo}), && \text{sin llamada} \\ &\quad L(x_{\{i:\}} \mid \text{prefijo} + [c \rightarrow \emptyset])) && \text{con la llamada pero sin resultado} \end{aligned}$$

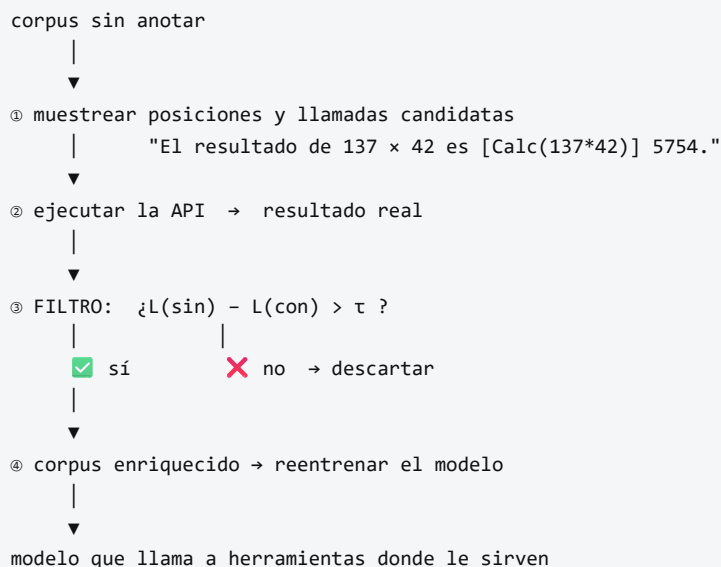
Se conserva la llamada si:

$$L^- - L^+ > \tau$$

L es la pérdida ponderada de predecir los tokens siguientes. τ es el umbral de filtrado.

El segundo término de L^- importa: compara contra **hacer la llamada sin recibir respuesta**, lo que aísla el valor del **resultado** frente al valor de la mera presencia del texto de la llamada.

6. Arquitectura o flujo



7. Qué observar en el paper original

- El **criterio de filtrado** y su justificación: es el corazón del método y ocupa poco espacio en el artículo.
- La **comparación de L^-** contra dos alternativas (sin llamada y con llamada sin resultado). Ese detalle evita conservar llamadas que solo ayudan por el texto que introducen.
- La **tabla de rendimiento por herramienta**: cuánto aporta cada una en qué benchmark. No todas aportan igual.
- El **análisis de escala**: el método necesita un modelo suficientemente capaz para generar llamadas plausibles; por debajo de cierto tamaño no funciona.
- La comprobación de que **no se degrada** la capacidad de modelado de lenguaje del modelo base.

8. Evidencia y resultados

El modelo base es un GPT-J de 6 700 millones de parámetros, enriquecido con el procedimiento y reentrenado.

Resultados evaluados en modo zero-shot sobre tareas donde cada herramienta debería ayudar: aritmética (calculadora), preguntas de conocimiento (buscador y sistema de QA), tareas multilingües (traductor) y preguntas sensibles a la fecha (calendario).

Toolformer mejora sustancialmente sobre el modelo base del mismo tamaño y, en varias de estas tareas, alcanza o supera a modelos mucho mayores **sin** herramientas, manteniendo su capacidad de modelado de lenguaje.

Las cifras por benchmark y herramienta están en las tablas del artículo. Verificarlas allí antes de citarlas.

La miniatura de este eje aísla el criterio de filtrado: de tres candidatas, sobrevive la que reduce la pérdida por encima de τ ; la llamada absurda, que la **aumenta**, se descarta sola.

9. Impacto

- Establece que el uso de herramientas puede **aprenderse**, no solo prompt-earse. Es un cambio conceptual respecto a ReAct.
- Introduce un criterio automático y barato de utilidad, reutilizable en otros contextos.
- Anticipa la normalización del *tool calling* como capacidad nativa de los modelos, en lugar de como capa externa.
- Su lógica sobrevive a su implementación: hoy el acceso a herramientas se estandariza con protocolos, pero la pregunta «¿esta llamada aporta valor medible?» sigue siendo la correcta.

10. Limitaciones

1. **Enseña cuándo llamar, no garantiza que la herramienta acierte.** Un Toolformer conectado a una API que devuelve basura aprenderá a llamarla con confianza.

2. **τ no tiene valor universal.** Bajo, conserva llamadas inútiles (coste y latencia); alto, descarta llamadas útiles.
3. **Coste de construcción del corpus:** hay que muestrear muchas candidatas y ejecutarlas todas.
4. **Cada llamada es independiente.** No hay composición: no aprende a encadenar dos herramientas para un resultado intermedio.
5. **Depende de la escala del modelo base:** por debajo de cierto tamaño, las llamadas candidatas no son suficientemente buenas.
6. **La reducción de pérdida es una aproximación de la utilidad**, no la utilidad misma. Puede premiar correlaciones espurias.
7. **Sin gestión de errores:** no aborda qué hacer cuando la API falla o tarda.

11. Errores comunes

Error	Corrección
«Toolformer garantiza respuestas correctas»	Garantiza que la llamada ayudó a predecir el texto . La corrección del resultado depende de la herramienta.
«Más llamadas a herramientas = mejor agente»	Cada llamada cuesta latencia y dinero. La métrica útil es la utilidad marginal por llamada, no el conteo.
«Es lo mismo que ReAct»	ReAct usa herramientas en el prompt; Toolformer aprende dónde insertarlas y reentrena con ello.
«Necesita anotadores humanos»	No. El criterio es la propia pérdida del modelo: por eso el título dice <i>teach themselves</i> .
«El filtro compara solo con no llamar»	Compara también contra llamar sin recibir respuesta, para aislar el valor del resultado.

12. Relación con trabajos anteriores

- **P13 ReAct (2022)** — usar herramientas dentro del bucle.
- **P10 GPT-3 (2020)** — aprendizaje en contexto, que permite generar las llamadas candidatas con pocos ejemplos.
- **WebGPT (Nakano et al., 2021)** — navegación con supervisión humana; el contraste directo con la autosupervisión de Toolformer. [arXiv:2112.09332](#)
- **PAL / Program-Aided Language Models (2022)** — delegar el cálculo a un intérprete. [arXiv:2211.10435](#)

13. Relación con trabajos posteriores

- **Tool calling nativo** en las APIs de modelos comerciales: la funcionalidad se vuelve producto.
- **Model Context Protocol** — estandarización del descubrimiento y la invocación de herramientas. [modelcontextprotocol.io](#)
- **P16 Sistemas agentic** — herramientas tipadas, permisos, presupuesto y seguridad de la cadena de suministro.
- **Trabajo sobre composición de herramientas (2023+)** — encadenar llamadas, que este paper no aborda.

14. Notebook asociado

P14_toolformer.ipynb

Qué implementa: el criterio de filtrado sobre tres candidatas (útil, marginal y absurda), un barrido de τ que muestra el compromiso entre conservar de más y de menos, y métricas que sustituyen al conteo bruto de llamadas.

Qué NO implementa: modelo de lenguaje, muestreo de candidatas, ejecución real de APIs ni reentrenamiento. Las pérdidas están fijadas para exhibir el criterio, y así se declara en la salida.

```
ai-evolution paper-lab P14 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe el criterio de filtrado y explica qué compara.
Explicar	Explica por qué comparar contra «llamada sin resultado» es necesario.
Aplicar	Barre τ en el notebook y anota cuántas llamadas sobreviven en cada caso.
Analizar	Diseña un caso donde la reducción de pérdida premie una llamada inútil.
Evaluar	Un equipo mide el éxito de su agente por «llamadas a herramientas por respuesta». Evalúa la métrica y propón tres alternativas.
Crear	Define una herramienta nueva para un dominio que conozcas y describe qué texto del corpus revelaría su utilidad.

16. Autoevaluación

1. ¿Qué señal sustituye a la anotación humana y por qué está disponible gratis?
2. ¿Qué pasa con τ muy bajo? ¿Y muy alto?
3. ¿Por qué el método necesita un modelo base suficientemente grande?
4. ¿Qué **no** garantiza este método sobre las respuestas de las herramientas?
5. ¿Por qué el conteo de llamadas es una mala métrica de calidad?
6. ¿En qué se diferencia de ReAct, en una frase?
7. ¿Qué capacidad relacionada con herramientas queda fuera del alcance de este paper?

17. Respuestas esperadas

1. La reducción de la pérdida de predecir el texto siguiente. Está disponible en cualquier corpus sin etiquetar, porque el «objetivo» es el propio texto que ya venía después.
2. Con τ bajo se conservan llamadas inútiles: el modelo aprende a invocar herramientas constantemente, con su coste y su latencia. Con τ alto se descartan llamadas útiles y el modelo vuelve a inventar los datos.

3. Porque las llamadas candidatas se generan con pocos ejemplos en el prompt. Si el modelo no produce llamadas plausibles, el filtro no tiene nada bueno que conservar.
4. Que sean correctas. El criterio mide utilidad predictiva, no veracidad.
5. Porque no distingue competencia de bucle. Siete llamadas pueden ser una descomposición excelente o un reintento caro que no converge.
6. ReAct **usa** herramientas mediante prompting; Toolformer **aprende** dónde llamarlas y reentrena el modelo con ese conocimiento.
7. La composición: encadenar la salida de una herramienta como entrada de otra. Cada llamada se evalúa de forma independiente.

18. Fuentes primarias

- Schick, T. et al. (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*. **NeurIPS 2023**. arXiv:2302.04761 · consultado 2026-08-16.
- Nakano, R. et al. (2021). *WebGPT: Browser-assisted question-answering with human feedback*. arXiv:2112.09332 · consultado 2026-08-16.
- Gao, L. et al. (2022). *PAL: Program-aided Language Models*. arXiv:2211.10435 · consultado 2026-08-16.



Evaluación — P14 · Toolformer: los modelos de lenguaje pueden enseñarse a sí mismos a usar herramientas

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Toolformer: Language Models Can Teach Themselves to Use Tools* (2023, arXiv:2302.04761 · NeurIPS 2023) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

- (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
- (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

- (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
- (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

- (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

- (10) Ejecuta `P14_toolformer.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
- (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
- (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

- (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
- (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).